FRC PROGRAMMING CURRICULUM — MODULE 1, LESSON 02

Template Structure + File Roles

Walk through every generated file and understand why it exists before touching anything.

Instructor Edition — Not for student distribution

At a Glance

Field	Detail
Target audience	HS freshmen & sophomores, new to FRC
Hardware	None — VSCode and simulator only
Session length	3 hours
Key tools	VSCode command palette, WPILib sim, AdvantageScope

Learning Objectives

- Students can name every file in the WPILib template and state its job in one sentence
- Students can use the WPILib command palette to build, simulate, and create new classes
- Students can launch the simulator, connect AdvantageScope, and drive using the keyboard or a game controller
- Students can explain why LoggedRobot is used instead of TimedRobot
- Students can move a hardcoded value into Constants.java and verify nothing breaks
- Students can add a logged output and find it by name in AdvantageScope
- Students can find and fix at least two bugs in a broken robot project
- Students can match a bug symptom to the file it lives in

Before You Start

Room setup

- VSCode open on projector with a freshly generated XRP template project
- BearBots project (bearbots-xrp-code) cloned and open on all student machines (or USB drive distributed)
- AdvantageScope available on instructor laptop and all student laptops (bundled with WPILib)
- Digital handout open in browser: lesson-02-vscode-template.html
- Broken robot lab seeded project loaded and ready (provided separately)
- Orbit Odyssey PDF open and queued: pdfs/orbit-odyssey-manual_V1.pdf

The most important instructor mindset for this lesson

Resist the urge to skip the file tour. Students who don't understand the template will spend the rest of the season editing the wrong files. 20 minutes of foundation here prevents hours of confusion later.

Bypass code for Part 8 (Orbit Odyssey unlock)

Bypass code: ORBIT! — entered in the instructor bypass field on Part 7 (Symptom Match). Use this if a student can't complete Symptom Match due to technical issues, or to unlock for the whole class at the end of the lab. Do not give students this code unprompted.

Session Timing

Time	Phase	Part	You do	Students do
0–5 min	Hook	—	Scroll file tree silently. Ask: what is all this?	Explore the file tree. Guess what each file does.
5–30 min	Git + project setup	Part 1	Walk through Git install (online or offline), clone, and the three Git commands.	Install Git if needed, clone the BearBots repo, run git --version to confirm.
30–60 min	File tour	Part 2	Walk through each file. Click expandable cards on projector. Step through the boot sequence diagram.	Follow along. Ask questions. Note anything surprising. Use the brain prompt before switching parts.
60–80 min	Simulator demo	Part 3	Live demo: launch sim, walk through GUI panels, connect AdvantageScope, drive with keyboard. Walk through XRP hardware port table.	Replicate each step. Don't move on until they've driven and watched a value move in AdvantageScope.
80–100 min	Read the data	Part 4	Run Graph Racing. Drive hidden patterns. Call out interesting moments.	Set up graphs. Watch encoders. Predict maneuvers. Closest prediction wins candy.
100–130 min	Hands-on challenges	Part 5	Circulate. Facilitate. Don't give answers.	Activity 1 (File Prediction game), Activity 2 (Scavenger Hunt — all 6), Activity 2b (Add Your Name).
130–155 min	Broken robot lab	Part 6	Circulate. Redirect with questions, not answers. Ask "what does the symptom tell you?"	Find and fix 4 bugs independently. Read the symptom before clicking anything.
155–165 min	Symptom Match	Part 7	Circulate. Encourage working from memory first. Use bypass if time is critical.	Match each bug symptom to its file. All 4 correct unlocks Part 8.
165–180 min	Orbit Odyssey	Part 8	Introduce the game. Read scoring table. Run brainstorm if time permits.	Read scoring. Pair brainstorm: what does the robot need to do?

If running short

Cut Graph Racing (Part 4) first, then Activity 1 (File Prediction) from Part 5. Scavenger Hunt, Add Your Name, the Broken Robot Lab, and Symptom Match should always happen. Part 8 (Orbit Odyssey) can be shortened to the scoring table read-only — skip the brainstorm.

Part 1 — Git + Project Setup (5–30 min)

Opening hook (0–5 min)

Open a fresh XRP template project on the projector. Don't say anything. Scroll the file tree slowly. Then:

Script — what to say

"VSCode just created these files. You didn't write any of them. Before we touch anything — does anyone want to guess what each one does?"

Wait for guesses. Even wrong guesses are useful. Validate the good ones. Defer the rest to Part 2.

Git install and clone (5–25 min)

Have students check Git first in the VS Code terminal:

Check for Git

Open the terminal: Ctrl+` (backtick) or Terminal → New Terminal

Type: git --version

If you see a version number — Git is installed. Skip to clone.

If you see 'git is not recognized' — follow the install steps below.

Use the swap toggle in the handout to switch between install paths:

- Online: git-scm.com → download installer → run with defaults → restart VS Code terminal
- Offline: USB drive contains the installer — same steps without the download

5th-grade version: what is Git?

Git is a time machine for your code. Every time you commit, you save a snapshot you can return to. If you break something at 11pm, you can go back to the version that worked. On a team, it lets two people work on the same project without overwriting each other.

The three commands students will use every session:

Command	When
git add .	After making changes — stages all modified files
git commit -m "message"	Saves a snapshot with a description
git pull	Start of every session — gets latest updates from GitHub

On git push

Push uploads commits to GitHub so teammates can see them. Students will do this later. For now, they commit locally — work is saved and tracked even without pushing. Don't go deep on push today.

After clone

Have students open the BearBots project (bearbots-xrp-code) in a VS Code window. Keep the XRP-Template open in a second window — they'll compare both in Part 2.

Part 2 — The Files (30–60 min)

Walking through each file

For each file: open on projector, explain job in one sentence, explain what breaks if missing, explain the most common mistake. Students use the expandable file cards in the handout to follow along.

Main.java

- Job: entry point. Calls RobotBase.startRobot(Robot::new). That's all.
- Missing: robot doesn't start. Not even a compile error — just silence.
- Most likely mistake: adding code here. The comment says not to. People do it anyway.

> **5th-grade version**
>
> Main.java is the ignition switch. It exists so the JVM has somewhere to start. After it fires startRobot(), it steps aside forever. You never add code here.

Robot.java

- Job: lifecycle manager. Contains robotPeriodic(), autonomousInit(), teleopInit(), etc.
- XRP-Template: extends TimedRobot. BearBots: extends LoggedRobot. That's Diff Card 1 in the handout.
- Most important line: CommandScheduler.getInstance().run() in robotPeriodic(). Without it, no commands ever execute.
- Most likely mistake: removing the CommandScheduler call, or putting subsystem logic here instead of in a subsystem class.

RobotContainer.java

- Job: the switchboard. Creates subsystems. Wires buttons to commands. Provides auto command.
- Missing: robot builds but no subsystems exist. Nothing responds to input.
- Most likely mistake: putting subsystem logic here instead of in subsystem classes.

Constants.java

- Job: number vault. Motor IDs, speeds, PID gains. XRP-Template ships it nearly empty.
- Missing: magic numbers scattered across five files. One wiring change breaks everything.
- Key point: this isn't the only constants file. Real teams split: TunerConstants, FieldConstants, ArmConstants. That's a design decision. BearBots uses inner classes as a middle ground.
- Most likely mistake: not using it at all.

build.gradle

- Job: build recipe. Specifies WPILib version, vendor libraries, compile and deploy config.
- Edited by: the WPILib tools (Manage Vendor Libraries), not by hand.
- Most likely mistake: editing it manually and breaking version alignment.

ExampleSubsystem.java / ExampleCommand.java

- Template scaffolding. No real hardware, no real logic. Deleted in BearBots project.
- Replaced by DriveSubsystem.java and actual robot files. Covered in Lesson 3.

Six diff cards

Walk through each diff card in Part 2 of the handout. For each: find the difference in the BearBots project before reading the card.

Card	File	Change
1	Robot.java	extends TimedRobot → extends LoggedRobot
2	Robot.java	Logger setup block added to constructor
3	RobotContainer.java	IO injection pattern — new DriveIOXRP() passed to subsystem
4	RobotContainer.java	Default command wired up
5	Constants.java	Inner class structure — DriveHardware, etc.
6	build.gradle	AdvantageKit vendor dependency added

The LoggedRobot vs TimedRobot question

Students will ask. LoggedRobot extends TimedRobot. Same lifecycle methods, same timing. AdvantageKit adds a logging wrapper around each cycle. The command framework is opt-in through CommandScheduler.run() in robotPeriodic().

Boot sequence diagram

The handout has an interactive boot sequence diagram. Each node is clickable. Walk through it on the projector:

- Main.java → RobotBase.startRobot()
- Robot constructor → Logger.start() → new RobotContainer()
- RobotContainer → instantiates subsystems with IO → wires bindings
- Every 20ms: robotPeriodic() → Logger.processInputs() → CommandScheduler → outputs → Logger.recordOutput()

Common questions to expect

"Can I put everything in Robot.java?" — Yes. Works for two weeks. Then 800 lines, merge conflicts, and everyone cries.

"What's the vendordeps folder?" — JSON files telling the build system where to download vendor libraries.

"What happens if code takes longer than 20ms?" — Loop overrun. Sensor readings go stale, motor updates drift. Never put Thread.sleep() or file I/O in periodic methods.

Brain prompt before moving on

"Without looking: if a command doesn't run when you press a button, which file do you check first?"

Answer: RobotContainer first (binding lives there), then confirm CommandScheduler.getInstance().run() is in robotPeriodic().

Part 3 — The Simulator (60–80 min)

Do this live on the projector. Students follow on their own laptops. Go slowly — every step matters.

Step 1 — Switch to BearBots project

Close the XRP-Template project. Open bearbots-xrp-code\Lesson01\First Drive. The XRP-Template has no logging — AdvantageScope connects but shows nothing. BearBots has AdvantageKit wired up so students get real data immediately.

Step 2 — Launch the simulator

- WPILib hexagon → WPILib: Simulate Robot Code → Enter
- Check halsim_gui → OK
- Wait for build. Simulation GUI opens automatically.

Step 3 — Understand the GUI panels

Panel	Purpose
Robot State	Enable/disable. Switch Teleop/Auto/Test modes. Forgetting to enable is the #1 student mistake.
Joysticks	Map keyboard or controller to robot joystick slots. Drag Keyboard 0 to Joystick[0].
System Console	Print statements, errors, stack traces. Check here first when nothing works.
NetworkTables	Live values from robot code. AdvantageKit data appears as "AdvantageKit/…"

"Nothing is happening" almost always means:

Robot not enabled, or there's an error in System Console. Check both before assuming the code is wrong.

Step 4 — Connect AdvantageScope

- Open AdvantageScope
- File → Connect to Simulator (Windows: Ctrl+Shift+K)
- Left panel populates — look for AdvantageKit/ folder
- Connect BEFORE enabling the robot — don't miss first-cycle data

Step 5 — Set up keyboard driving

- Simulation GUI → Joysticks panel → drag Keyboard 0 to Joystick[0]
- Robot State → Teleoperated → Enable
- Click back on Simulation GUI window (keyboard only works when focused)
- Default keys: W forward, S backward, A turn left, D turn right

If a student has a real controller

Plug in before launching sim. Appears automatically in Joysticks panel. Drag to Joystick[0]. Move sticks — watch axis values update in the panel to confirm it's working.

Step 6 — XRP hardware port table

Walk through this before driving. Students need to know what each port maps to before they touch physical hardware.

Device	Port	Subsystem	Direction
Left drive motor	XRPMotor 0	DriveSubsystem	Output
Right drive motor	XRPMotor 1	DriveSubsystem	Output
Left encoder	DIO 4 / 5	DriveSubsystem	Input
Right encoder	DIO 6 / 7	DriveSubsystem	Input
Gyro	XRPGyro	DriveSubsystem	Input
Arm servo	XRPServo 4 → Servo 1	ArmSubsystem	Output

Right motor inversion

Per WPILib docs: right motor spins backward when positive output is applied. It must be inverted in code. This is already handled in DriveIOXRP.java. Bug 4 in the lab deliberately removes this inversion — students fix it there.

Common mistakes to watch for

Robot not enabled → nothing moves, students think code is broken

AdvantageScope not connected → graphs flat

Keyboard not working → Simulation GUI window not focused

Graphs jump to huge numbers → unit conversion issue (inches vs meters)

Part 4 — Read the Data (80–100 min)

Graph Racing: competitive enough to be fun. Teaches graph literacy as a side effect.

Setup

Everyone sets up the same two graphs in AdvantageScope:

- Expand Drive/ folder in left panel
- Drag LeftPositionMeters to graph area
- Drag RightPositionMeters onto the same graph tab
- Enable in Teleop, start driving

Prediction round

Before each maneuver, students predict what the graph will look like. Closest prediction wins candy.

Maneuver	What the graph should show
Drive straight forward 2 seconds	Both encoders increase equally
Spin in place 2 seconds	Encoders go opposite directions
Instructor's mystery pattern	Class describes what they see, then you reveal

"What is the robot doing?"

You drive. Students watch only the encoder graphs — no watching the 3D model window. The handout scaffolds the key insight: comparing left vs right encoders tells you rotation rate. That's odometry.

Facilitation notes

Keep turns short — 60–90 seconds each. After each run, ask the driver: "Did the graph match what you expected?"

The most interesting moments are mismatches — student expects symmetry, gets asymmetry. That's a conversation about the robot, not a failure.

Brain prompt before Part 5

"In your own words: how would you tell from encoder data alone whether the robot drove straight or turned? No code — just the concept."

Part 5 — Hands-On Challenges (100–130 min)

Three activities. Students work at their own pace. All have extensions for early finishers.

Activity 1 — File Prediction Game

Pairs. Two rounds. Tap-to-select (iPad compatible). Students tap a tile, then tap the target to place it.

Round 1 — File → Role:

Tile	Correct target
Main.java	First thing that runs on boot
Robot.java	Lifecycle manager
RobotContainer.java	Subsystems + button bindings
Constants.java	All the magic numbers
build.gradle	Vendor library config

Round 2 — Snippet → File:

Snippet	File
RobotBase.startRobot(Robot::new)	Main.java
CommandScheduler.getInstance().run()	Robot.java
new RobotContainer()	Robot.java constructor
controller.leftBumper().onTrue(...)	RobotContainer.java
static final double kMaxOutput = 1.0	Constants.java

Facilitation

Don't correct pairs during Round 1 — let them use the "Check answers" button and work out wrong answers themselves. Move on when the round clicks for most of the group.

Activity 2 — Constants Scavenger Hunt

Six magic numbers are hiding in the seeded project — hardcoded directly in files instead of Constants.java or relevant constant classes. Students find each one, move it, and rebuild.

Pre-hunt reference — teach before students start

Walk through the three reference boxes in the handout on the projector before releasing students to work independently.

Box 1 — How to declare a constant

The three-word recipe: **public static final**. Every constant in WPILib uses this pattern.

```
public static final double kSpeedLimit = 0.8;
public static final int    kLeftMotorPort = 0;
public static final String kProjectName = "BearBots";
```

Why all three? **public** — any file can read it. **static** — belongs to the class, not an instance; no new Constants() needed. **final** — locked after startup; can never be reassigned.

Constants live inside inner classes inside Constants.java. If the inner class doesn't exist yet, students create it — copy the public static class block and give it a new name.

5th-grade version

A constant is a sticky note on the wall that everyone can read but nobody can change. public means it's posted where everyone can see it. static means it belongs to the room, not any one person. final means it's laminated — you can read it but you can't write on it.

Box 2 — How to import and use a constant in another file

Add an import at the top of the file, then use the constant by name:

```
import frc.robot.Constants.DriveConstants;
xaxisSpeed = MathUtil.clamp(xaxisSpeed, -1.0, 1.0) * DriveConstants.kSpeedLimit;
```

No import needed if using the fully-qualified name: **Constants.DriveConstants.kSpeedLimit**

VS Code quick-fix import — demo this before the hunt starts

When a student types a constant name that hasn't been imported yet, VS Code underlines it red. Hover it (or press **Ctrl+.**) and choose **Add import**. VS Code writes the import line automatically. This is the one shortcut worth demoing on the projector before students start.

Box 3 — The k naming rule

All WPILib constants start with lowercase **k**, followed by UpperCamelCase. This convention appears across every WPILib source file. Students who use it look like they've been writing robot code for years.

Examples by category:

Speeds/power: kSpeedLimit, kMaxTurnSpeed, kAutodriveSpeed
Angles: kMaxAngleDeg, kStowedAngleDeg, kScoringAngleDeg
Distances: kWheelCircumferenceMeters, kTrackWidthMeters
Timing: kDriveTimeSeconds, kTurnTimeSeconds
Ports/IDs: kLeftMotorPort, kRightMotorPort, kControllerPort
Strings: kProjectName, kAutoDefault

Rules to teach explicitly:

Always start with k — speedLimit ✗ → kSpeedLimit ✓
Include the unit when ambiguous — kMaxAngle ✗ → kMaxAngleDeg ✓
No abbreviations unless obvious — kWC ✗ → kWheelCircumferenceMeters ✓
Drop the class name from the constant — DriveConstants.kDriveSpeed ✗ → DriveConstants.kSpeed ✓
Booleans: use kIs or kEnable prefix — kMotorInverted ✗ → kIsRightMotorInverted ✓

Why the unit rule matters

WPILib uses it everywhere: kMaxVelocityMetersPerSecond, kWheelRadiusMeters. A function expecting radians and receiving degrees gives you a robot that overshoots by 57×. The unit in the name is a free sanity check every time anyone reads the code.

The six hunts:

Hunt	What	Where hardcoded	Where it moves
1	Speed limit 0.8	Drive.java	DriveConstants.kSpeedLimit
2	Joystick deadband 0.1	RobotContainer.java	OperatorConstants.kDeadband
3	Max arm angle 120	Arm.java setAngle()	ArmConstants.kMaxAngleDeg (change from 180.0 → 120.0)
4	Wheel circumference 0.1885	DriveConstants.java	DriveConstants.kWheelCircumferenceMeters using Math.PI * kWheelDiameterMeters
5	Delay times 2.0 and 1.3	AutonomousTime.java	New AutoConstants inner class in Constants.java
6	Project name string "Activity2"	Robot.java	Constants.kProjectName

Hunt 5 is the hardest

Two related magic numbers, a new inner class, and multiple call sites. Use as Silver/Gold challenge or skip for struggling students.

Instructor moves

Walk the room. Don't point at the bug — ask "where does that value get used?"

If a student breaks the build: "Good. What does the error say? Which file? Which line?" Walk them through the import or reference fix. This is a valuable mistake.

If a student finishes all 6: direct to extensions (loop count, uptime timer, boolean, favorite number — all in robotPeriodic()).

Extensions (if finished early)

All go in robotPeriodic() in Robot.java:

- Loop count: add private int loopCount = 0; field, then loopCount++; Logger.recordOutput("LoopCount", loopCount);
- Uptime timer: Logger.recordOutput("UptimeSeconds", Timer.getFPGATimestamp());
- Boolean: Logger.recordOutput("RobotAlive", true); — ask: what does a boolean look like in AdvantageScope?
- Favorite number: Logger.recordOutput("FavoriteNumber", 42);

Activity 2b — Add Your Name to the Robot

First time a student's code produces visible output in a real data stream.

The task

1. In Constants.java, add: public static final String PILOT_NAME = "Your Name";
2. In Robot.java inside robotPeriodic(), add: Logger.recordOutput("Pilot", Constants.PILOT_NAME);
3. Run the simulator. Open AdvantageScope. Find Pilot in the log tree.

Why this works

Every student sees their name appear in a robot data stream for the first time. The concept — anything you log shows up in AdvantageScope — becomes concrete and personal before they need it for debugging.

Extension: ask students to log a timestamp or random number. Watching a value update live every 20ms is more memorable than any explanation of the logging loop.

Part 6 — Broken Robot Lab (130–155 min)

Four robots. Four different files. Each compiles and launches — then something goes wrong. Students read the symptom, find the bug, fix it.

> **Redirect with questions, not answers.**
> "What does the symptom tell you about where the problem is?" not "look at line 5."

Bug 1 — Robot.java: extends TimedRobot instead of LoggedRobot

Symptom	Simulator starts, robot drives, AdvantageScope shows no log data at all. AdvantageKit/ folder never appears.
Fix	public class Robot extends LoggedRobot {
Why	TimedRobot doesn't know about AdvantageKit. The logger attaches hooks to LoggedRobot's loop cycle. Without it, no inputs are captured.

Bug 2 — RobotContainer.java: joystick axes swapped

Symptom	Pushing joystick forward makes the robot spin. Pushing left makes it drive forward. Controls feel 90° rotated.
Buggy code	() -> -controller.getLeftX() labeled // forward, () -> -controller.getLeftY() labeled // turn
Fix	Swap them — Y axis is forward/backward, X axis is turn. The comments label them correctly; the arguments are reversed.

Bug 3 — Constants.java: kMaxOutput = 100.0 instead of 1.0

Symptom	Slightest joystick input = instant full speed. No proportional control.
Fix	public static final double kMaxOutput = 1.0;
Why	Motor output is clamped to [-1, 1]. Multiplying by 100 immediately saturates the clamp. The motor sees 1.0 the instant the joystick moves.

Bug 4 — DriveSubsystem.java: left motor not inverted

Symptom	Driving "forward" makes robot spin in place. Left encoder goes negative while right goes positive.
Buggy code	Only rightMotor.setInverted(true) — left motor runs same direction as right
Fix	leftMotor.setInverted(true); rightMotor.setInverted(true);
Why	Left and right motors face opposite directions on the chassis. Both must be inverted to push forward together on the XRP.

> **Note on Bug 4 and XRP motor ports**
> The lab code shows XRPMotor leftMotor = new XRPMotor(1) and XRPMotor rightMotor = new XRPMotor(0). Per WPILib docs: XRPMotor 0 = Left, XRPMotor 1 = Right. The right motor is inverted in the actual DriveIOXRP.java. The bug here is specifically the missing left motor inversion, not the port numbers.

Part 7 — Symptom Match (155–165 min)

Unlock gate for Part 8. Students match each bug symptom to the file it came from. All 4 correct unlocks Orbit Odyssey.

Correct matches

Symptom	File
"AdvantageKit/ folder never appears in AdvantageScope"	Robot.java
"Push joystick forward → robot spins. Push left → drives forward."	RobotContainer.java
"Slightest joystick input = instant full speed"	Constants.java
"Encoders go opposite directions when driving straight"	DriveSubsystem.java

Unlock logic

- Student gets all 4 correct → Part 8 unlocks automatically (saved to localStorage)
- Student can't complete it → use instructor bypass code: ORBIT!
- Once unlocked, Part 8 stays unlocked on that browser/device

Facilitation

Encourage students to work from memory first — no scrolling back to the lab. This is a recall exercise.

If a student is truly stuck after trying, let them peek at one bug to unblock.

Part 8 — Orbit Odyssey (165–180 min)

First time students see the competition they're building toward. Keep it energizing — don't turn it into a lecture.

> **Instructor mindset**
> The goal is curiosity, not comprehension. Students don't need to understand every rule today. They need to leave asking "how would I score points?" That question drives the next five lessons.

Setup

- Orbit Odyssey PDF open on projector (linked from Part 8 of the handout)
- Scoring table visible in the handout

Scoring table

Action	Points
Rubble in Low Zone	1
Rubble in High Goal	3
Ping pong ball in Low Zone	2
Ping pong ball in High Goal	5
Large ball on High Pedestal	10
Robot parked in Low Zone	3

No penalties. No element holding limits. No robot inspection. Focus on scoring.

Format: 1v1 round-robin — every student competes against every other student.

"Going Further" brainstorm (if time permits)

Run if you have 10+ minutes remaining. Sets up Lesson 3's subsystem introduction.

Round	Prompt (2 min pairs)
Round 1	List every physical action the robot needs to do to score rubble in the High Goal. Don't say "pick it up" — describe the specific movements.
Round 2	Which actions could the robot do on its own during autonomous? Which ones require a human driver? Why?
Round 3	If you had to organize all those robot actions into groups — mechanisms that work independently — how many groups would there be? What would you call them?

Share out (5 min): each pair shares top 2–3 requirements. Write on the whiteboard without editing. Let duplicates stack. Don't resolve debates.

> **Script — teaser for Lesson 03**
> *"Keep that list in your head. Next lesson we talk about subsystems — how robots break their jobs into separate files. When we do that whiteboard, you're going to recognize these exact tasks. The code structure we build over the next five lessons is a direct answer to what you just wrote down."*

> **Do NOT reveal the BearBots robot design here.**

That reveal happens at the end of Lesson 03 after students have worked through the design themselves.

Instructor Edition — Not for student distribution